

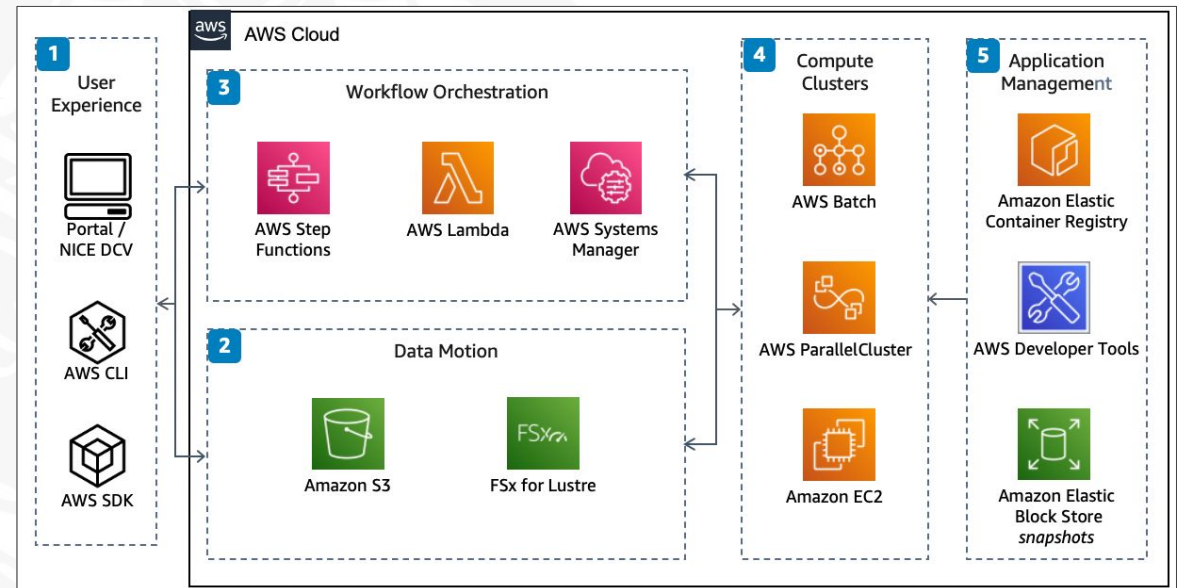


CyberDojo : The School of
Cyberdefense

SAA-C03
Module 3:
Design High
Performing
Architectures

Module 3: Design High-Performing Architectures

- Section 3.1: High-Performing Storage (EBS, EFS, FSx, S3 performance)
- Section 3.2: High-Performing Compute (EC2 families, Auto Scaling, Lambda)
- Section 3.3: High-Performing Databases (RDS, Aurora, DynamoDB, ElastiCache)
- Section 3.4: High-Performing Networking (CloudFront, Global Accelerator)
- Section 3.5: Data Ingestion & Transformation (Kinesis, DataSync, Glue, EMR)



SAA-C03

Section 3.1: High-Performing and Scalable Storage Solutions

- **Module 3:** Design High-Performing Architectures (24% of exam)
- **Section 3.1:** High-Performing and Scalable Storage Solutions
- **Topics Covered:**
 - Storage Types Overview (Block, File, Object)
 - Amazon EBS Deep Dive (Volume Types, Performance)
 - Amazon EFS (Performance Modes, Throughput Modes)
 - Amazon FSx (Lustre for HPC, Windows for Enterprise)
 - Amazon S3 Performance (Transfer Acceleration, Multipart Upload)
 - Hybrid Storage (Storage Gateway)



SAA-C03

Storage Types Overview - Block vs. File vs. Object

Three Storage Types (MEMORIZE differences):

Aspect	Block (EBS)	File (EFS, FSx)	Object (S3)
Access	Single EC2 instance	Multiple EC2 instances	Anywhere (HTTP/HTTPS)
Protocol	iSCSI	NFS, SMB	REST API (HTTP)
Performance	Highest (microsecond latency)	High (millisecond latency)	Good (millisecond latency)
Use case	Databases, OS boot	Shared files, web serving	Backups, static content, data lakes
Scaling	Manual (resize volumes)	Automatic	Automatic
Pricing	Pay for provisioned	Pay for used	Pay for stored + requests

- **SAA Exam Focus:**
 - **EBS** = Single instance, high performance (databases)
 - **EFS** = Multiple instances, Linux workloads (shared content)
 - **FSx** = Specialized workloads (HPC, Windows)
 - **S3** = Unlimited storage, any access pattern

SAA-C03



Amazon EBS Deep Dive - Volume Types

EBS Volume Types:

Type	Family	Use Case	Max IOPS	Max Throughput
gp3	SSD	General purpose (default choice)	16,000	1,000 MB/s
gp2	SSD	General purpose (older, gp3 is better)	16,000	250 MB/s
io2 / io1	SSD	High performance (databases)	256,000	4,000 MB/s
io2 Block Express	SSD	Largest, fastest	256,000	4,000 MB/s
st1	HDD	Throughput optimized (big data)	500	500 MB/s
sc1	HDD	Cold storage (infrequent access)	250	250 MB/s



Amazon EBS Deep Dive - Volume Types

gp3 vs. gp2 (Important!):

Feature	gp3	gp2
Baseline IOPS	3,000 (any size)	Based on size (3 IOPS/GB)
Baseline throughput	125 MB/s	250 MB/s (at 1,000+ IOPS)
Cost	Cheaper (20% less than gp2)	More expensive
When to choose	Most workloads (default)	Legacy or specific needs

- **SAA Exam Focus:**

- **gp3** = New default EBS volume (cheaper, better baseline)
- **io2** = Provisioned IOPS for critical databases (SAP HANA, Oracle)
- **st1** = Big data, logs, data warehouses (throughput matters, not IOPS)



EBS Performance Optimization

EBS Performance Tips:

Tip	Why
Use gp3 as default	Better baseline, cheaper
Provisioned IOPS (io2) for databases	Consistent, predictable performance
EBS-optimized instances	Dedicated bandwidth
EBS Snapshots	Incremental, schedule during off-peak
EBS Fast Snapshot Restore (FSR)	No warm-up time (costs extra)



Amazon EFS - Performance Modes & Throughput Modes

EFS Performance Modes:

Mode	Max IOPS	Use Case
General Purpose	~50,000 IOPS	Most workloads (web servers, CMS)
Max I/O	~500,000+ IOPS	Large-scale, highly parallel (big data, analytics)

Bursting Mode Details:

Size	Baseline Throughput	Burst Throughput
1 GB	0.05 MiB/s	100 MiB/s
1 TB	50 MiB/s	100 MiB/s
10 TB	500 MiB/s	500 MiB/s (no burst)

EFS Throughput Modes:

Mode	How it works	Best for
Bursting	Baseline + burst credits	Variable workloads (web servers)
Provisioned	Set throughput (MiB/s)	Predictable, high throughput
Elastic	Automatically scales	Unpredictable workloads (default for new)

- **SAA Exam Focus:**

- **General Purpose** = Default (most workloads)
- **Max I/O** = Big data, analytics (massive parallelism)
- **Elastic Throughput** = New default (auto-scales, pay for used)



SAA-C03

Amazon FSx - Lustre and Windows

FSx for Lustre (High Performance Computing):

Aspect	Details
What it is	Managed Lustre filesystem (open-source HPC)
Use cases	ML training, genomics, seismic analysis, financial modeling
Performance	Sub-millisecond latency, hundreds GB/s throughput
Integration	Can link to S3 (data repository)
Deployment	Scratch (temp data, faster) vs. Persistent (long-term)

FSx for Windows File Server:

Aspect	Details
What it is	Managed Windows file server (SMB protocol)
Use cases	Windows applications, user home drives, legacy apps
Features	Active Directory integration, DFS, deduplication
Performance	Up to 2 GB/s throughput, 100K IOPS



Amazon FSx - Lustre and Windows

FSx Comparison:

Feature	Lustre	Windows
Protocol	Lustre (custom)	SMB (Windows native)
OS support	Linux	Windows, Linux (SMB client)
Use case	HPC, ML	Enterprise Windows apps
S3 integration	Yes (native)	No (use Storage Gateway)

- **SAA Exam Focus:**
 - **FSx Lustre** = HPC, ML, scientific computing (keywords: "genomics", "seismic", "simulation")
 - **FSx Windows** = Enterprise Windows workloads (keywords: "Active Directory", "SMB", "home drives")



SAA-C03

Amazon S3 Performance Optimization

S3 Performance Limits (per prefix):

Operation	Requests per second
GET/HEAD	5,500 per second
PUT/POST/DELETE	3,500 per second

S3 Transfer Acceleration:

Aspect	Details
What it does	Uses edge locations to speed up uploads
Best for	Cross-region uploads (far from bucket)
Cost	Pay per GB (additional to standard transfer)
When to use	Upload from far regions, poor internet

How to Increase Performance:

Strategy	How it works	Example
Add prefixes	Distribute load across more prefixes	logs/ 2024/01/, logs/ 2024/02/
Use parallel requests	Multiple simultaneous connections	10 threads = 10x throughput
S3 Transfer Acceleration	Use edge locations for uploads	Cross-region uploads
Multipart Upload	Break large files into parts	Files >100 MB (recommended)
Byte-range fetches	Download specific ranges	Parallel downloads



SAA-C03

Amazon S3 Performance Optimization

Multipart Upload:

Aspect	Details
What it does	Split file into parts, upload in parallel
Recommended for	Files >100 MB
Required for	Files >5 GB
Max parts	10,000
Part size	5 MB to 5 GB

- **SAA Exam Focus:**

- **Add prefixes** = Simplest way to increase performance
- **Multipart upload** = Required for large files
- **Transfer Acceleration** = Cross-region uploads (exam loves this)



Hybrid Storage - AWS Storage Gateway

Hybrid Storage - AWS Storage Gateway
Three Gateway Types (MEMORIZE):

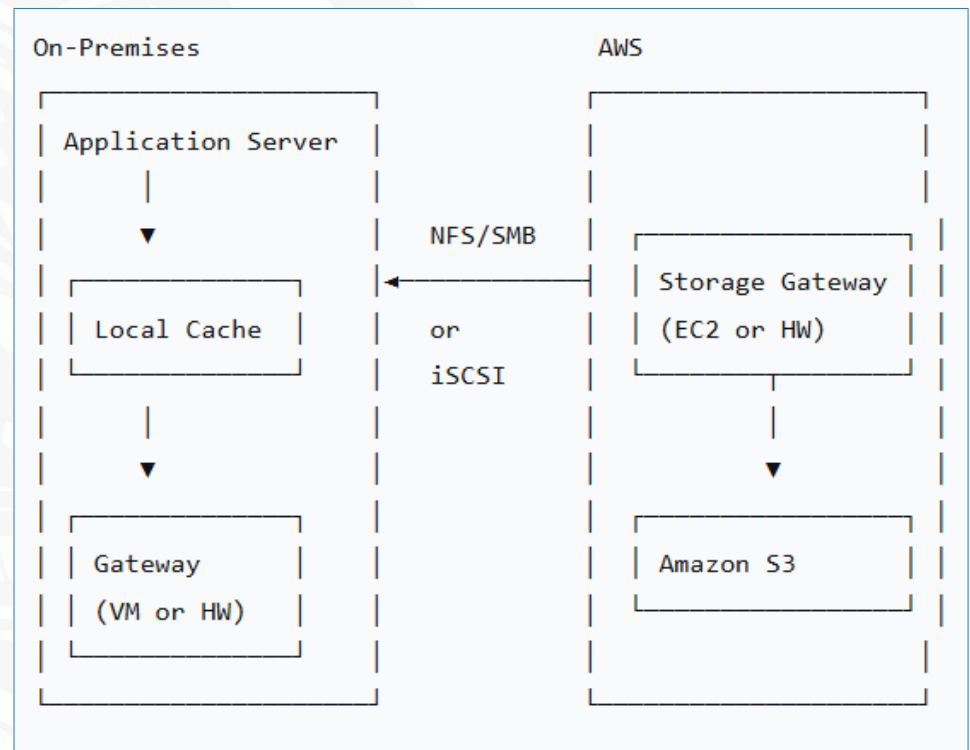
Type	Protocol	Presents as	Use Case
File Gateway	NFS, SMB	File share	On-prem file storage to S3
Volume Gateway	iSCSI	Block volume	Backup, DR to EBS snapshots
Tape Gateway	iSCSI (VTL)	Virtual tape library	Replace physical backup tapes

• **SAA Exam Focus:**

- **File Gateway** = On-premises files to S3 (lowest latency access)
- **Volume Gateway** = iSCSI block storage, backups to EBS snapshots
- **Tape Gateway** = Replace physical tape backups (VTL)
- **Cache** = Frequently accessed data stays on-prem (low latency)

SAA-C03

File Gateway Architecture:



Lab/Hands-on Guide - Storage Performance

Lab 1: Compare EBS Volume Types

1. Launch 3 EC2 instances (same type)
2. Attach: gp3, io2, st1 volumes
3. Run fio benchmark (random read/write)
4. Compare IOPS and throughput

Lab 2: EFS Configuration

1. Create EFS filesystem
2. Mount on 2 EC2 instances (different AZs)
3. Write file from Instance A
4. Read file from Instance B (verify shared)
5. Test General Purpose vs. Max I/O modes

Lab 3: S3 Transfer Acceleration

1. Create S3 bucket with Transfer Acceleration enabled
2. Upload large file from remote location
3. Compare with/without acceleration
4. Use AWS CLI: `--endpoint-url https://bucket.s3-accelerate.amazonaws.com`

Lab 4: S3 Multipart Upload

1. Use AWS CLI to upload 1 GB file
2. Command: `aws s3 cp largefile s3://bucket/ --multipart-chunksize 64MB`
3. Verify parallel upload in CloudTrail



Section 3.1 Summary - Storage Solutions

Service	Key Performance Feature	Exam Focus
EBS gp3	3,000 baseline IOPS (any size)	Default choice, cheaper than gp2
EBS io2	Provisioned IOPS up to 256K	Critical databases
EBS Multi-Attach	Shared volume (io1/io2 only)	Cluster applications
EFS General Purpose	~50K IOPS	Most workloads
EFS Max I/O	~500K IOPS	Big data, analytics
FSx Lustre	HPC performance	ML, genomics, simulations
FSx Windows	SMB protocol	Enterprise Windows
S3 Transfer Acceleration	Edge locations for upload	Cross-region uploads
S3 Multipart Upload	Parallel upload parts	Files >100 MB
Storage Gateway	Hybrid on-prem cache	Low-latency access



Section 3.2: High-Performing and Elastic Compute Solutions

- Topics Covered:
 - EC2 Instance Families (General, Compute, Memory, Storage, Accelerated)
 - Instance Family Selection Guide
 - EC2 Auto Scaling Policies
 - AWS Auto Scaling (Multi-Resource)
 - AWS Lambda Performance (Memory, Concurrency)
 - Container Orchestration (ECS, EKS, Fargate)
 - AWS Batch
 - Amazon EMR



EC2 Instance Families - Complete Overview

Five Instance Families (MEMORIZE for SAA):

Family	Series	Use Case	Characteristics
General Purpose	T, M	Web servers, dev environments	Balanced CPU/Memory/Network
Compute Optimized	C	Batch processing, gaming servers	High CPU performance
Memory Optimized	R, X, Z	In-memory databases, caches	Large RAM capacity
Storage Optimized	I, D	NoSQL databases, data warehouses	High IOPS, dense storage
Accelerated Computing	P, G, Inf	ML, graphics rendering	GPUs, FPGAs, specialized hardware

- **SAA Exam Focus:**

- **T series** = Burstable CPU (credits accumulate)
- **M series** = Default choice (don't know what to pick)
- **C series** = CPU-heavy (keywords: "batch", "gaming", "ad serving")
- **R series** = Memory-heavy (keywords: "in-memory DB", "caching", "SAP")



SAA-C03

EC2 Instance Families - Complete Overview

Family	Specific Use Case	Example Instance Types
T (Burstable)	Low-cost, intermittent workloads	t4g, t3, t2
M (General)	Balance of resources	m7g, m6g, m5
C (Compute)	CPU-intensive	c7g, c6g, c5
R (Memory)	Memory-intensive (general)	r7g, r6g, r5
X (Memory)	Large memory (SAP HANA)	x2idn, x2iedn
Z (Memory)	High compute + memory	z1d
I (Storage)	High IOPS (NVMe SSD)	i4i, i3
D (Storage)	Dense storage (HDD)	d3, d2
P (GPU)	ML training, HPC	p4d, p3
G (GPU)	ML inference, graphics	g5, g4dn
Inf (Inferentia)	ML inference (cost-effective)	inf2, inf1



Instance Family Selection Guide - Real World Scenarios

Scenario-Based Selection (Exam Favorite):

Scenario	Instance Family	Why
Web server (variable traffic)	T3 (Burstable)	Cost-effective, burst credits
Web server (steady traffic)	M6g (General)	Balanced, predictable
Batch processing job	C6g (Compute)	High CPU, fast processing
In-memory Redis cache	R6g (Memory)	Large RAM capacity
MySQL database (not RDS)	R6g (Memory)	Database needs memory
SAP HANA	X2idn (High Memory)	Massive memory (up to 24 TB)
NoSQL database (Cassandra)	I4i (Storage)	High local NVMe IOPS
ML training (model training)	P4d (GPU)	Multiple GPUs, fast interconnect
ML inference (prediction)	Inf2 (Inferentia)	Cost-effective inference
Video rendering	G5 (GPU)	Graphics rendering



EC2 Auto Scaling Policies - Deep Dive

Four Scaling Policy Types (MEMORIZE):

Policy Type	How it works	Best for	Complexity
Target Tracking	Maintain metric at target (e.g., CPU=50%)	Simplicity, dynamic workloads	Low
Simple Scaling	Add/remove fixed number when alarm triggers	Predictable patterns	Low
Step Scaling	Add/remove based on alarm severity	Granular control	Medium
Scheduled Scaling	Scale at specific times	Predictable patterns (lunch rush)	Low

Target Tracking Example (Most Common):

Policy: Keep average CPU at 50%
When CPU > 60%: Add instances (scale out)
When CPU < 40%: Remove instances (scale in)
Cooldown: 300 seconds between scaling actions

Step Scaling Example:

Alarm Severity	Action
CPU > 70% for 2 min	Add 2 instances
CPU > 90% for 1 min	Add 5 instances
CPU < 30% for 5 min	Remove 2 instances

Scheduled Scaling Example:

Monday-Friday, 9 AM: Scale to 10 instances
Monday-Friday, 5 PM: Scale to 2 instances
Weekends: Scale to 1 instance



AWS Auto Scaling - Multi-Resource Scaling

What is AWS Auto Scaling? Automatically scales multiple resources simultaneously

Resources AWS Auto Scaling Can Manage:

Resource	What it scales
EC2 Auto Scaling Groups	EC2 instances
DynamoDB	Read/write capacity units
Aurora	Aurora Replicas (read scaling)
ECS	Container instances
Spot Fleet	Spot Instance requests

SAA Exam Focus:

- **AWS Auto Scaling** = Multi-resource, predictive scaling (ML)
- **EC2 Auto Scaling** = EC2 only (still most common)
- Predictive scaling = Exam keyword for "anticipates traffic patterns"



AWS Lambda Performance - Concurrency

Lambda Concurrency:

Concept	Details
Account limit	1,000 concurrent executions per region (soft limit)
Reserved concurrency	Guarantee capacity for critical functions
Provisioned concurrency	Keep functions "warm" (no cold start)

Provisioned Concurrency:

Aspect	Details
What it does	Pre-initializes execution environments
Cost	Pay for provisioned environments (even idle)
When to use	Latency-sensitive applications
Example	API Gateway → Lambda (needs sub-100ms response)



Container Orchestration - ECS vs. EKS vs. Fargate

Container Orchestration Comparison (MEMORIZE):

Service	Best for	Control plane	Worker nodes
ECS (EC2 launch)	AWS-native, cost optimization	Managed (free)	You manage EC2
ECS (Fargate launch)	Serverless containers	Managed (free)	AWS manages (serverless)
EKS (EC2 launch)	Kubernetes expertise	Managed (\$0.10/hr)	You manage EC2
EKS (Fargate launch)	Serverless Kubernetes	Managed (\$0.10/hr)	AWS manages (serverless)

When to choose which:

Scenario	Best choice	Why
Simple web app on AWS	ECS + Fargate	Serverless, easy
Need Kubernetes for portability	EKS	Multi-cloud or hybrid
Large, steady workloads	ECS + EC2 launch type	Cheaper (no Fargate premium)
Batch/spot workloads	ECS + EC2 (Spot)	Up to 90% discount
Already using Kubernetes on-prem	EKS	Consistent experience



Container Orchestration - ECS vs. EKS vs. Fargate

Fargate Benefits:

Benefit	Why it matters
No EC2 management	No patching, scaling, or provisioning
Pod-level isolation	Each pod has its own security boundary
Pay per vCPU-hour	No idle cluster costs
Automatic scaling	Scales with workload

- SAA Exam Focus:

- **Fargate** = Serverless containers (keywords: "no server management", "fully managed")
- **EKS** = Kubernetes (keywords: "multi-cloud", "Kubernetes", "portability")
- **ECS** = AWS-native (keywords: "deep AWS integration", "simple")



AWS Batch - Managed Batch Processing

What is AWS Batch? Fully managed service for running batch computing jobs at scale

How AWS Batch Works:

- User submits job → AWS Batch → Provisions EC2 instances (as needed) → Runs jobs → Terminates instances

SAA Exam Focus:

- **AWS Batch** = Batch computing (keywords: "batch", "scientific simulation", "financial modeling")
- Runs on **EC2 and Spot instances** (cost optimization)
- **Managed compute environment** = Let AWS handle scaling

Key Concepts:

Concept	Description
Job queues	Jobs wait in queue until resources available
Compute environments	Managed or unmanaged EC2 fleet
Job definitions	Container image, vCPU, memory, command
Job scheduling	Priority-based, dependency-aware



Amazon EMR - Managed Big Data Platform

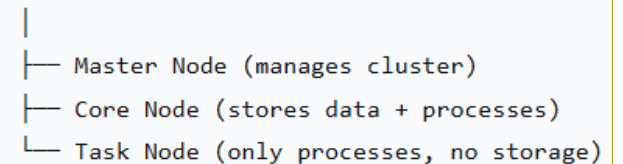
What is EMR? Managed big data platform running Hadoop, Spark, Hive, HBase on EC2 clusters

EMR Use Cases:

Use Case	Description
ETL (Extract, Transform, Load)	Prepare data for analytics
Log processing	Parse server logs, extract insights
Data transformation	Convert formats, clean data
Machine learning	Data preparation, model training
Scientific simulations	Genomics, physics, financial

EMR Architecture:

Data Source (S3) → EMR Cluster (Spark) → Target (S3/Redshift)



• SAA Exam Focus:

- **EMR** = Big data processing (keywords: "Hadoop", "Spark", "petabyte", "ETL")
- **EMR Serverless** = No cluster management (pay per job)
- **Spot Instances** = Major cost savings (exam loves this)



Compute Services Comparison - Decision Tree

START: What is your workload?

- ├ Long-running server application?
 - ├ Predictable, steady traffic? → EC2 + ASG (On-Demand)
 - ├ Variable, spiky traffic? → EC2 + ASG (Target Tracking)
 - └ Need Kubernetes? → EKS
- ├ Short-running, event-driven? (<15 min)
 - └ Lambda (serverless)
- ├ Containerized application?
 - ├ Want serverless? → ECS + Fargate
 - ├ Want control over EC2? → ECS + EC2 launch type
 - └ Need Kubernetes portability? → EKS
- ├ Batch processing (hours)?
 - ├ Running once? → AWS Batch
 - ├ Recurring? → AWS Batch (scheduled)
 - └ Big data (Hadoop/Spark)? → EMR
- └ ML workload?
 - ├ Training (GPU needed)? → EC2 P4d (or SageMaker)
 - └ Inference (prediction)? → Lambda or EC2 G5/Inf2

Quick Reference Table:

Workload	Best Service	Why
Web server (steady)	EC2 + ASG	Cost-effective, control
Web server (spiky)	Lambda + API Gateway	Pay per request
Microservices	ECS + Fargate	Serverless containers
Batch (short)	Lambda	No idle cost
Batch (long)	AWS Batch	Managed queues
Big data (Spark)	EMR	Managed Hadoop/Spark
ML training	SageMaker (or EC2 P4d)	Managed training
ML inference	Lambda or EC2 Inf2	Cost-effective



SAA-C03

Lab/Hands-on Guide - Compute Performance

Lab 1: EC2 Instance Family Comparison

1. Launch 3 EC2 instances (T3.micro, M5.large, C5.large)
2. Run CPU stress test: `stress --cpu 8 --timeout 60`
3. Compare CPU utilization and performance
4. Test burst credits on T3 (unlimited mode)

Lab 2: Auto Scaling Target Tracking

1. Create Launch Template
2. Create Auto Scaling Group (min=1, max=5)
3. Create Target Tracking policy (CPU=50%)
4. Generate CPU load using stress utility
5. Watch ASG add instances automatically

Lab 3: Lambda Memory vs. Performance

1. Create Lambda function (compute-intensive)
2. Test with 128 MB memory → measure duration
3. Increase to 512 MB → measure duration
4. Increase to 1769 MB → measure duration
5. Graph memory vs. duration (linear relationship!)

Lab 4: ECS + Fargate Container Deployment

1. Create ECS cluster (Fargate)
2. Create task definition (nginx container)
3. Create service (2 tasks)
4. Test load balancing across tasks
5. Scale service to 5 tasks (automatic)



Section 3.3: High-Performing Database Solutions

- **Topics Covered:**
 - Database Types Overview (Relational, NoSQL, In-Memory)
 - Amazon RDS Performance (Instance types, Provisioned IOPS)
 - Amazon Aurora Performance (5x MySQL, Aurora Replicas)
 - Aurora Serverless
 - Amazon DynamoDB Performance (Partition key design, DAX)
 - DynamoDB Accelerator (DAX) - In-memory cache
 - Amazon ElastiCache (Redis vs. Memcached)
 - Caching Strategies (Lazy loading, Write-through, TTL)
 - RDS Proxy (Connection pooling)
 - Read Replicas vs. Multi-AZ



Database Types Overview - Relational vs. NoSQL vs. In-Memory

Three Database Types (MEMORIZE):

Type	AWS Service	Use Case	Key Feature
Relational (SQL)	RDS, Aurora	Complex queries, transactions, joins	ACID compliance
NoSQL	DynamoDB, DocumentDB	Massive scale, simple lookups, flexible schema	Single-digit ms latency
In-Memory	ElastiCache	Caching, session storage, real-time analytics	Microsecond latency

When to choose which:

Requirement	Choose
Complex JOINS, ACID transactions	RDS or Aurora
Massive scale, predictable latency	DynamoDB
Speed up existing database	ElastiCache
JSON documents, MongoDB API	DocumentDB
Graph data (social networks)	Neptune
Time-series (IoT sensors)	Timestream



Amazon RDS Performance

RDS Performance Features:

Feature	What it does	When to use
Provisioned IOPS	Guarantee IOPS	I/O-intensive workloads
Storage Auto-Scaling	Automatically increase storage	Unpredictable growth
Multi-AZ	Synchronous standby (HA)	Production HA (not performance)
Read Replicas	Offload read traffic	Read-heavy workloads

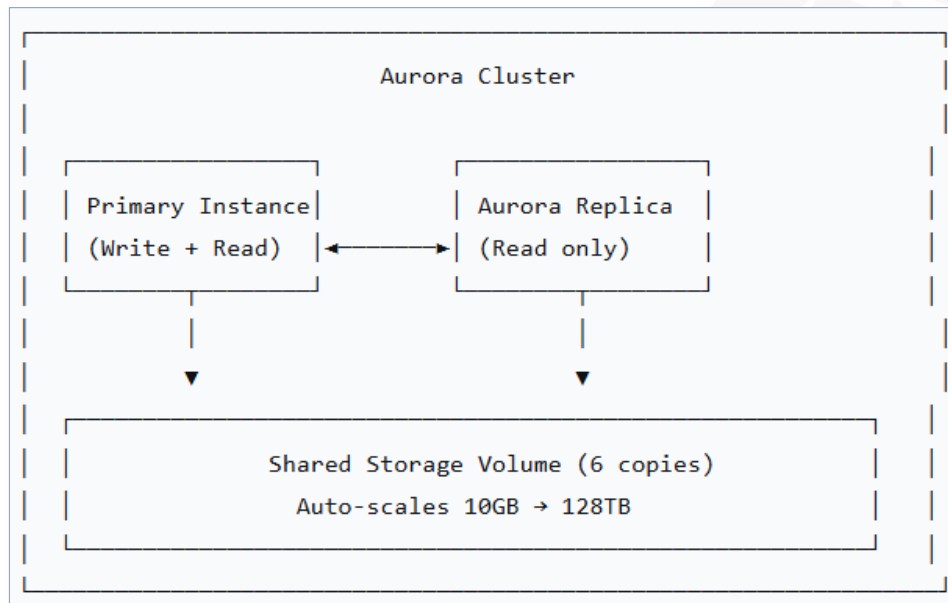
- **SAA Exam Focus:**

- **Provisioned IOPS** = Answer for "consistent, predictable IOPS"
- **Storage Auto-Scaling** = Avoid "storage full" errors
- **Read Replicas** = Performance (scaling reads)
- **Multi-AZ** = HA (failover), NOT performance



Amazon Aurora Performance - 5x MySQL, 3x PostgreSQL

Aurora Architecture (Key for SAA):



Aurora Features:

Feature	Benefit
Aurora Replicas (15)	Read scaling, failover targets
Auto-scaling storage	No manual provisioning
Fast failover	<30 seconds (promote replica)
Backtrack	Rewind database to previous point in time
Global Database	Cross-region replication (<1 sec)

- **5x MySQL, 3x PostgreSQL** = Memorize these numbers!
- **15 replicas** (vs. RDS 5 replicas)
- **6 copies across 3 AZs** = Extreme durability
- **Global Database** = Cross-region DR (1 sec RPO, 1 min RTO)



SAA-C03

Aurora Serverless - For Intermittent Workloads

What is Aurora Serverless? Aurora database that automatically scales compute capacity

Aurora Serverless vs. Provisioned Aurora:

Factor	Serverless	Provisioned
Capacity planning	None (automatic)	Choose instance type
Idle cost	Low (can scale to zero)	Pay for instance (even idle)
Cold start	Possible (few seconds)	None (always running)
Use case	Intermittent, dev/test	Production, predictable

When to use Aurora Serverless:

Scenario	Recommendation
Development/test environments	✓ Perfect (scale to zero at night)
Intermittent workloads (hourly batch)	✓ Great (no idle cost)
Unpredictable traffic spikes	✓ Good (auto-scales)
Production with consistent traffic	✗ Use provisioned (no cold start)

Aurora Serverless = Intermittent workloads (keywords: "development", "testing", "unpredictable")

- **Provisioned Aurora** = Production, predictable (keywords: "consistent", "steady-state")



Amazon DynamoDB Performance - Partition Key Design

DynamoDB Performance Model (CRITICAL for SAA):

Concept	How it works
Partition	Unit of storage (10 GB max per partition)
Partition key	Determines which partition item goes to
Throughput per partition	3,000 RCU or 1,000 WCU max
Hot partition	Single partition getting most traffic

Partition Key Design (Most Important!):

Poor Design (Hot Partition)	Good Design (Even Distribution)
<code>user_id</code> (1 user has 90% of data)	<code>user_id + timestamp</code> (more unique)
<code>status</code> (only 'active', 'inactive')	<code>order_id</code> (unique per order)
<code>date</code> (today's date only)	<code>YYYY-MM-DD-HH</code> (hour granularity)



DynamoDB Accelerator (DAX) - In-Memory Cache

What is DAX? In-memory cache for DynamoDB (microsecond latency)

DAX vs. ElastiCache for DynamoDB:

Feature	DAX	ElastiCache
Compatibility	DynamoDB API (drop-in)	Custom code
Latency	Microseconds	Microseconds
Use case	Cache DynamoDB reads	Cache anything
Complexity	Low (no code changes)	High (custom logic)

When to use DAX:

Scenario	Recommendation
Read-heavy DynamoDB workload	✓ Perfect
Need microsecond latency	✓ Great
Cannot change application code	✓ Drop-in replacement
Write-heavy workload	✗ Not helpful (cache misses)

- **DAX** = In-memory cache FOR DynamoDB (not generic)
- **Microsecond latency** = Key exam keyword for DAX



Amazon ElastiCache - Redis vs. Memcached

ElastiCache Overview: In-memory caching service for speeding up applications

When to choose Redis:

Scenario	Why Redis
Need to persist cache data	Disk snapshots
Complex data structures	Lists, sets, sorted sets
High availability	Multi-AZ + replicas
Session storage	Persistence matters

When to choose Memcached:

Scenario	Why Memcached
Simple key-value caching	No complex structures
Massive scale (multi-threaded)	Uses multiple CPU cores
Can lose cache data	No persistence needed
Object caching	Simple get/set operations

- **SAA Exam FOCUS:**

- **Redis** = Default choice (more features)
- **Memcached** = Simple, multi-threaded, no persistence
- Exam often tests "when to choose Redis vs. Memcached"



Caching Strategies - Lazy Loading, Write-Through, TTL

Three Caching Patterns (MEMORIZE for SAA):

1. Lazy Loading (Cache-Aside) - Most Common:

- Read request → Check cache → Cache miss → Read DB → Store in cache → Return data
- Write request → Update DB → (cache NOT updated, now stale)

2. Write-Through:

- Write request → Update cache → Update DB → Return success
- Read request → Check cache → Cache hit → Return data

3. TTL (Time to Live):

- **What it does:** Cache entries expire after set time

Caching Pattern Decision:

Workload	Best Pattern
Read-heavy, can tolerate stale data	Lazy loading + TTL
Write-heavy, need consistency	Write-through
Critical data, must be accurate	Write-through + short TTL

• SAA Exam Focus:

- **Lazy loading** = Most common, simple, cache miss penalty
- **Write-through** = Consistent, higher write latency
- **TTL** = Automatic expiration (always use!)



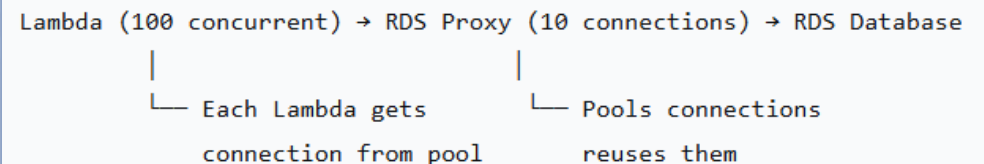
RDS Proxy - Connection Management

What is RDS Proxy? Managed connection pooling service for RDS and Aurora

Problem RDS Proxy Solves:

Problem	Without RDS Proxy	With RDS Proxy
Connection storms	Lambda functions overwhelm database	Proxy pools connections
Open connections limit	Hitting max connections	Proxy manages limit
Lambda cold start	Opening DB connection on each invoke	Reuse pooled connections

How RDS Proxy Works:



- **SAA Exam Focus:**

- **RDS Proxy** = Lambda + RDS (exam loves this pair!)
- Prevents connection storms from Lambda
- IAM authentication = No hardcoded passwords



Read Replicas vs. Multi-AZ - Performance vs. HA

Comparison Table (MEMORIZE - heavily tested):

Feature	Read Replicas	Multi-AZ
Primary purpose	Read scalability (performance)	High availability (failover)
Replication type	Asynchronous	Synchronous
Standby accepts traffic?	Yes (read traffic)	No (standby only)
Automatic failover?	No (manual promotion)	Yes (automatic)
Max number	5 (RDS), 15 (Aurora)	1 standby
Cross-region	Yes	No (same region)
Use case	Read-heavy workloads	Production HA

When to use which:

Requirement	Solution
"Need to scale read traffic"	Read Replicas
"Need automatic failover if AZ fails"	Multi-AZ
"Reporting queries slowing down OLTP"	Read Replica (offload reporting)
"Cannot afford any downtime"	Multi-AZ + Read Replicas (both!)
"Cross-region DR"	Read Replica (cross-region)

SAA Exam Focus:

- **Multi-AZ** = HA (keyword: "failover", "availability")
- **Read Replicas** = Performance (keyword: "read-heavy", "offload", "reporting")
- Both = HA + Performance



SAA-C03

Lab/Hands-on Guide - Database Performance

Lab 1: RDS Performance Comparison

1. Launch RDS (db.t3.micro, gp2 storage)
2. Launch RDS (db.t3.micro, gp3 storage)
3. Run sysbench OLTP test on both
4. Compare IOPS and latency
5. Upgrade to Provisioned IOPS → retest

Lab 2: Aurora vs. MySQL Performance

1. Launch RDS MySQL (db.r5.large)
2. Launch Aurora MySQL (db.r5.large)
3. Run same workload on both
4. Compare: throughput, latency, failover time
5. Add Aurora Replicas → test read scaling

Lab 3: DynamoDB Partition Key Design

1. Create table with poor partition key (status)
2. Load skewed data (90% 'active')
3. Monitor CloudWatch (ThrottledRequests)
4. Create table with good key (user_id)
5. Compare throttling

Lab 4: DAX Cache Performance

1. Create DynamoDB table
2. Create DAX cluster
3. Run query against DynamoDB (measure latency)
4. Run same query against DAX (measure latency)
5. Compare: milliseconds vs. microseconds



Section 3.3 Summary - Database Solutions

Service	Key Performance Feature	Exam Focus
RDS	Provisioned IOPS for consistent performance	"Predictable IOPS"
Aurora	5x MySQL, 3x PostgreSQL, 15 replicas	"5x performance", "6 copies"
Aurora Serverless	Auto-scaling compute, scale to zero	"Intermittent", "dev/test"
Aurora Global DB	Cross-region DR (<1 sec RPO)	"Global DR", "1 second"
DynamoDB	Partition key design (avoid hot partitions)	"Even distribution", "high cardinality"
DAX	Microsecond latency for DynamoDB	"DAX = DynamoDB cache"
ElastiCache Redis	Persistence, complex data types	Default caching choice
ElastiCache Memcached	Multi-threaded, simple, no persistence	"Simple", "massive scale"
RDS Proxy	Connection pooling for Lambda	"Lambda + RDS = RDS Proxy"
Read Replicas	Read scaling (performance)	"Offload reads", "reporting"
Multi-AZ	Automatic failover (HA)	"Failover", "availability"



Section 3.4: High-Performing and Scalable Network Architectures

- **Topics Covered:**

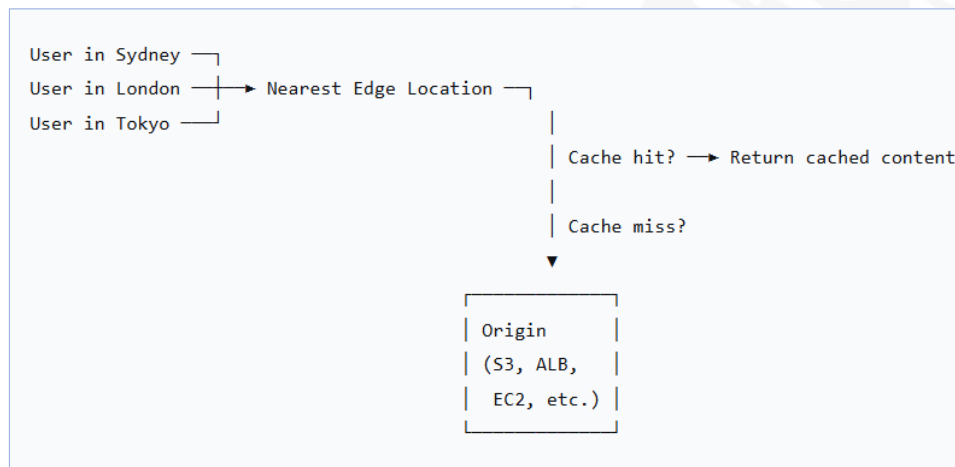
- Amazon CloudFront (CDN, caching behaviors, origin shield)
- CloudFront Security (signed URLs/cookies, WAF integration)
- AWS Global Accelerator (Anycast IPs, AWS backbone)
- CloudFront vs. Global Accelerator Comparison
- Load Balancing Deep Dive (ALB path/host routing, NLB static IP)
- Network Connectivity (VPC Peering vs. Transit Gateway)



Amazon CloudFront - CDN with Edge Locations

What is CloudFront? Content Delivery Network (CDN) that caches content at edge locations

CloudFront Architecture:



- **SAA Exam Focus:**

- **CloudFront** = HTTP/HTTPS only (not TCP/UDP)
- **Edge Locations** = Cache content (not compute, except Lambda@Edge)
- **Origin Shield** = Single origin request (reduces origin load)



CloudFront Security - Signed URLs, Signed Cookies, WAF

CloudFront Security Features:

Feature	What it does	Use Case
Signed URLs	Time-limited URLs for specific files	Paid content, private downloads
Signed Cookies	Time-limited access to multiple files	Subscription content (e.g., video streaming)
Field-Level Encryption	Encrypt sensitive data at edge	PII, credit cards, healthcare data
WAF Integration	Protect against web exploits	SQL injection, XSS, rate limiting

Signed URLs vs. Signed Cookies:

Feature	Signed URLs	Signed Cookies
Access scope	Single file	Multiple files
Use case	Paid downloads, single file access	Subscription content, video streaming
Implementation	URL parameter (? Expires=...&Signature=...)	Cookie header

- **SAA Exam Focus:**

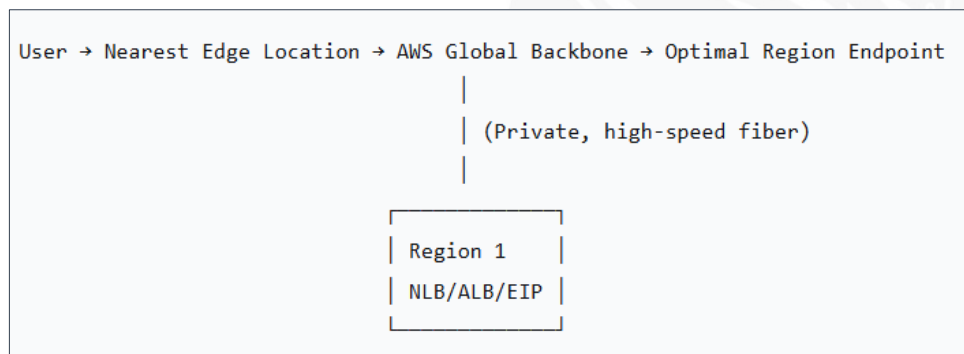
- **Signed URLs** = Paid content (pay-per-file)
- **Signed Cookies** = Subscription content (access to all files)
- **Field-Level Encryption** = Protect specific form fields at edge



AWS Global Accelerator - Anycast IPs, AWS Backbone

What is Global Accelerator? Network layer service that uses AWS global network to improve performance

How Global Accelerator Works:



Key Features

Feature	Benefit
Anycast IPs	Two static IP addresses (same globally)
AWS Backbone	Traffic travels on private AWS network (not internet)
Health checks	Automatic regional failover (<30 seconds)
TCP termination at edge	Reduced handshake latency
Port mapping	Multiple ports to multiple endpoints

Global Accelerator Endpoints:

- NLB (Network Load Balancer)
- ALB (Application Load Balancer)
- EC2 instances (Elastic IP)



SAA-C03

CloudFront vs. Global Accelerator - COMPARISON (CRITICAL)

Comparison Table (MEMORIZE - heavily tested):

SAA Exam Keywords:

Feature	CloudFront	Global Accelerator
Protocols	HTTP, HTTPS	TCP, UDP (any protocol)
Caching	☑ Yes (caches content)	✗ No (no caching)
Static IPs	✗ No (dynamic)	☑ Yes (2 Anycast IPs)
Use case	Cacheable content (static, API)	Dynamic content, real-time, TCP/UDP
Edge locations	400+ (caching)	400+ (entry points)
Path optimization	To edge only	To edge + backbone to origin
Best for	Websites, static assets, APIs	Gaming, VoIP, IoT, real-time

Keyword	Choose
"Cache", "CDN", "static", "edge caching"	CloudFront
"Real-time", "UDP", "gaming", "VoIP", "static IP"	Global Accelerator



SAA-C03

Load Balancing Deep Dive - ALB Path/Host Routing, NLB Static IP

ALB Advanced Routing (SAA loves this):

Routing Type	Example	Use Case
Path-based	/api/* → API service, /images/* → Image service	Microservices
Host-based	api.example.com → API service, www.example.com → Web service	Multi-tenant
Query string	?user=premium → Premium service	A/B testing
HTTP method	GET → Cache service, POST → DB service	REST APIs

SAA Exam Focus:

- **Path-based routing** = Most common ALB use case (microservices)
- **Sticky sessions** = Required for stateful apps (but avoid if possible)
- **NLB static IP** = Firewall whitelisting requirement



SAA-C03

Network Connectivity - VPC Peering vs. Transit Gateway

VPC Peering:

Aspect	Details
What it is	Direct connection between 2 VPCs
Transitive routing	✗ Not supported ($A \rightarrow B$, $B \rightarrow C \neq A \rightarrow C$)
IP overlap	✗ Not allowed (CIDRs must not overlap)
Scale	Limited (many peerings = complex)
Cost	Data transfer charges (same as AZ)

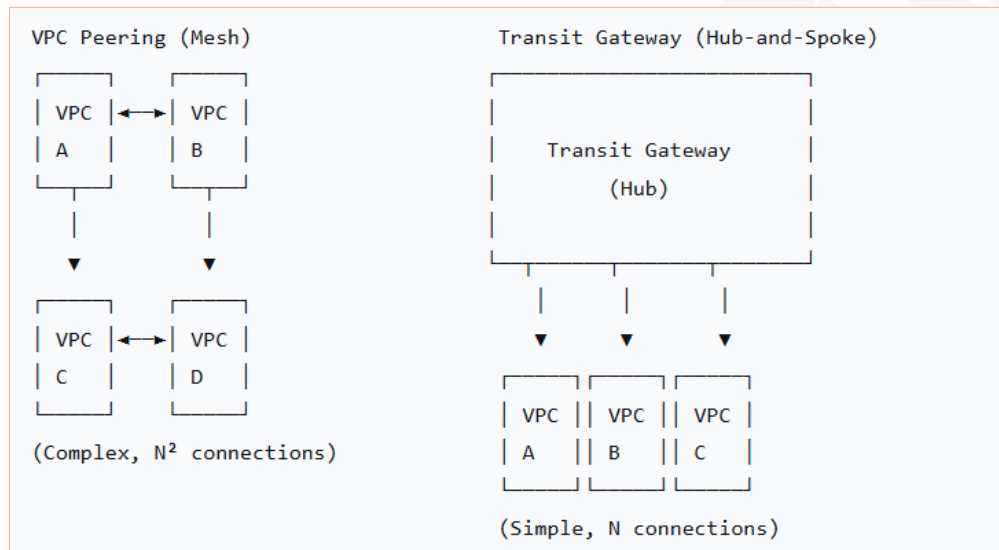
Transit Gateway (TGW):

Aspect	Details
What it is	Hub-and-spoke model for network connectivity
Transitive routing	✓ Supported (hub routes between spokes)
IP overlap	⚠ Possible (with route tables)
Scale	Thousands of attachments
Features	VPN, Direct Connect, peering, multicast



Network Connectivity - VPC Peering vs. Transit Gateway

VPC Peering vs. Transit Gateway:



- **SAA Exam Focus:**
 - **VPC Peering** = Simple, 2 VPCs, no overlapping CIDRs
 - **Transit Gateway** = Multiple VPCs, transitive routing, VPN aggregation
 - Exam scenario: "Connect 50 VPCs together" → Transit Gateway

SAA-C03



Section 3.4 Summary - Network Architectures

Service	Key Feature	Exam Focus
CloudFront	Edge caching (HTTP/HTTPS)	"Cache", "CDN", "static content"
CloudFront Signed URLs	Time-limited single file access	"Paid content", "pay-per-file"
CloudFront Signed Cookies	Time-limited multi-file access	"Subscription", "video streaming"
Global Accelerator	Anycast IPs, AWS backbone, TCP/UDP	"Static IP", "UDP", "gaming", "real-time"
ALB Path Routing	/api/* → different target group	"Microservices", "path-based"
ALB Host Routing	api.example.com → different TG	"Multi-tenant", "host-based"
ALB Sticky Sessions	Same user to same instance	"Stateful app", "shopping cart"
NLB Static IP	Fixed IP per AZ	"Firewall whitelist"
VPC Peering	Direct 2 VPC connection	"No transitive", "no overlap"
Transit Gateway	Hub-and-spoke for many VPCs	"50 VPCs", "transitive"



Section 3.5: High-Performing Data Ingestion & Transformation

- **Topics Covered:**

- Amazon Kinesis (Data Streams, Data Firehose, Data Analytics)
- Kinesis Data Streams Shards (throughput calculation)
- AWS DataSync (online data transfer)
- AWS Transfer Family (SFTP, FTPS, FTP)
- AWS Glue (serverless ETL, crawlers, Data Catalog)
- Amazon EMR (big data processing)
- Data Lakes on AWS (S3 + Lake Formation + Athena)



SAA-C03

Amazon Kinesis - Real-Time Data Streaming

What is Kinesis? Fully managed service for real-time processing of streaming data at massive scale

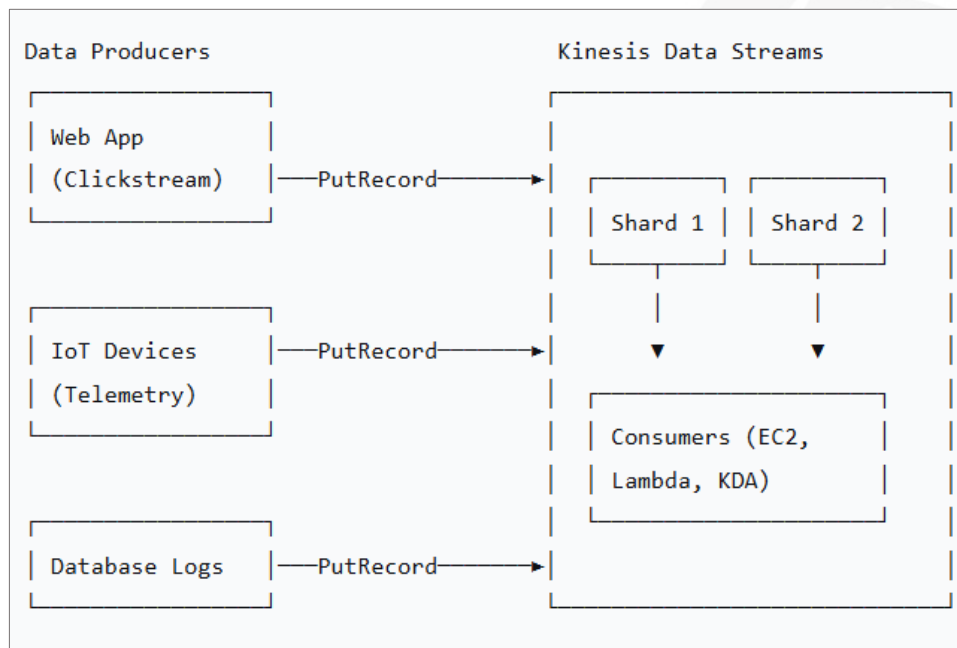
Kinesis Components (MEMORIZE):

Component	Purpose	Data Retention	Use Case
Kinesis Data Streams	Ingest and store streaming data	24 hours - 365 days	Real-time processing with custom code
Kinesis Data Firehose	Load streaming data to destinations	Near real-time (buffered)	S3, Redshift, OpenSearch, Splunk
Kinesis Data Analytics	Process streaming data with SQL or Flink	Real-time	Real-time dashboards, analytics
Kinesis Video Streams	Ingest video and audio	Configurable	Video analytics, ML, security cameras



Amazon Kinesis - Real-Time Data Streaming

Kinesis Data Streams Architecture:



- SAA Exam Focus:

- **Data Streams** = Real-time, custom processing (you write consumers)
- **Data Firehose** = Near real-time, fully managed delivery (no code)
- **Data Analytics** = Real-time SQL on streams (use Flink or SQL)



Kinesis Data Streams Shards - Throughput Calculation

What is a Shard? Unit of throughput in Kinesis Data Streams

Shard Throughput Limits (MEMORIZE):

Operation	Limit per Shard
Write (PUT)	1 MB/sec or 1,000 records/sec
Read (GET)	2 MB/sec (shared across consumers)
Enhanced Fan-Out Read	2 MB/sec per consumer (dedicated)

SAA Exam Focus:

- **Shard = Throughput unit** (not storage)
- **1 MB/sec write** = Per shard (hard limit)
- **Split shards** = Increase throughput (can't exceed limit per shard)



AWS DataSync - Online Data Transfer

What is DataSync? Online data transfer service to/from AWS (faster than internet)

DataSync Capabilities:

Source	Destination
On-premises (NFS, SMB)	S3, EFS, FSx
Another cloud (object storage)	S3, EFS, FSx
AWS (S3, EFS, FSx)	S3, EFS, FSx (different region/account)

- **SAA Exam Focus:**

- **DataSync** = Online data transfer (keywords: "migrate", "transfer", "sync")
- **DataSync Agent** = VM on-premises (required for on-prem sources)
- **Bandwidth throttling** = Control network impact



AWS Transfer Family - SFTP, FTPS, FTP

What is Transfer Family? Fully managed file transfer service supporting legacy protocols

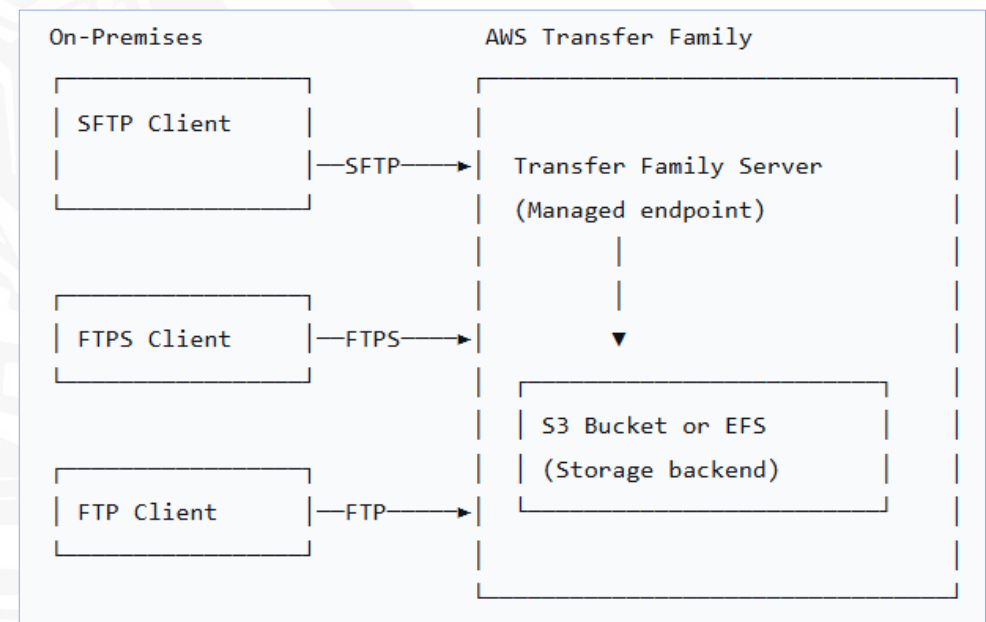
Supported Protocols:

Protocol	Port	Use Case
SFTP (SSH File Transfer)	22	Secure, modern
FTPS (FTP over SSL/TLS)	990 (explicit)	Legacy secure
FTP (File Transfer Protocol)	21	Legacy unencrypted

- **SAA Exam Focus:**

- **Transfer Family** = Legacy file transfer protocols (SFTP, FTPS, FTP)
- **Backend storage** = S3 or EFS (not EC2)
- **Use case** = Migrating from legacy FTP servers to AWS

Transfer Family Architecture:



SAA-C03

AWS Glue - Serverless ETL and Data Catalog

What is Glue? Serverless ETL (Extract, Transform, Load) service

When to use Glue (Exam Favorite):

Scenario	Why Glue
Serverless ETL	No Spark clusters to manage
Schema discovery	Crawlers automatically detect schema
Data Catalog	Central metadata for Athena, EMR, Redshift
PySpark/Spark	Complex transformations

Glue vs. EMR vs. Athena:

Service	Best for
Glue	Serverless ETL, schema discovery, Data Catalog
EMR	Large-scale, custom Spark/Hadoop clusters
Athena	Query S3 directly (no ETL)

- **Glue** = Serverless ETL (keywords: "ETL", "transform", "prepare data")
- **Glue Data Catalog** = Central metadata (used by Athena, Redshift Spectrum, EMR)

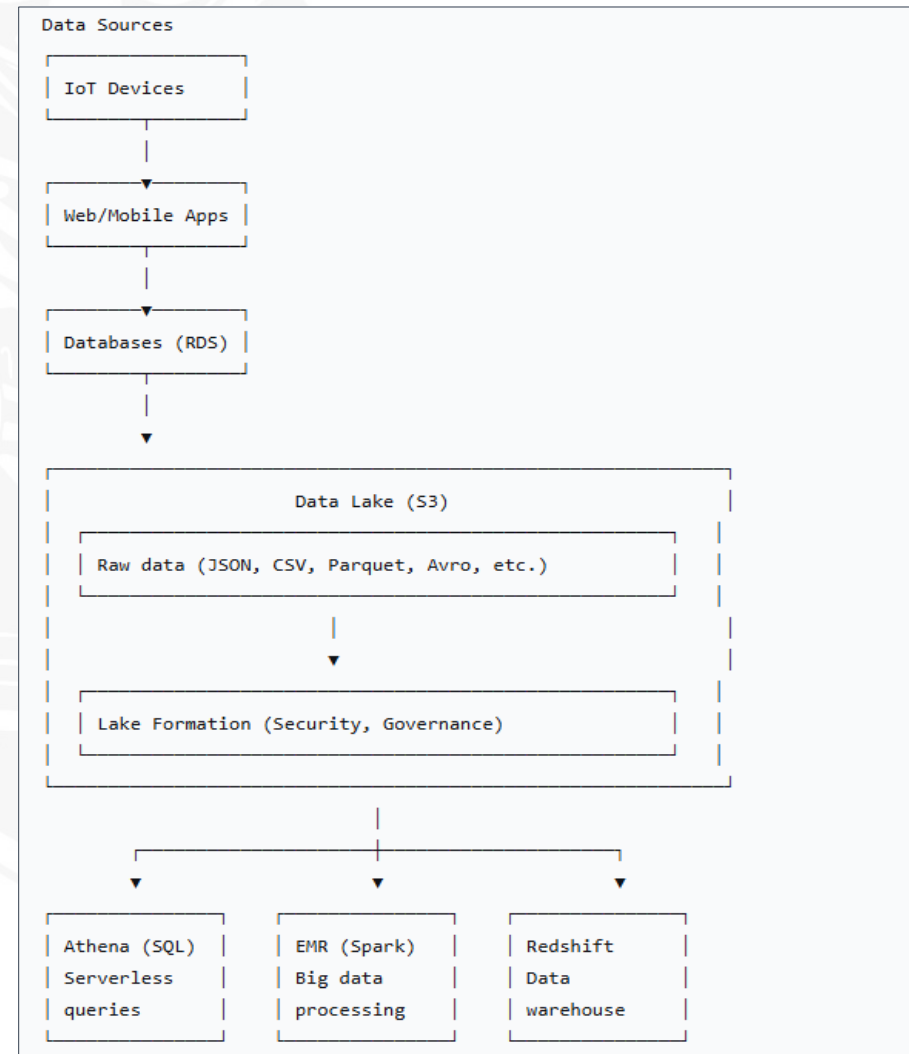


Data Lakes on AWS - S3 + Lake Formation + Athena

What is a Data Lake? Centralized repository for structured and unstructured data (store everything, analyze later)

Data Lake vs. Data Warehouse:

Feature	Data Lake	Data Warehouse
Data type	Raw, unstructured, semi-structured	Structured, processed
Schema	Schema-on-read (define when querying)	Schema-on-write (define before loading)
Use case	Data science, exploration, ML	BI, reporting, analytics
Cost	Lower (S3)	Higher (Redshift clusters)



SAA-C03

Data Lakes on AWS - S3 + Lake Formation + Athena

Lake Formation Features:

Feature	Benefit
Central permissions	Fine-grained access control (row/cell level)
Automated data discovery	Crawlers find data
Data filtering	Transform data (e.g., mask PII)
Encryption	Automatic (KMS)

- **SAA Exam Focus:**

- **Data Lake** = S3 (raw data)
- **Lake Formation** = Security + governance on data lake
- **Athena** = Query data lake directly (serverless SQL)



Section 3.5 Summary - Data Ingestion & Transformation

Service	Purpose	Key Exam Point
Kinesis Data Streams	Real-time data ingestion	Shard = 1 MB/s write, 2 MB/s read
Kinesis Data Firehose	Managed delivery to S3/Redshift	Buffer (size or time), no code
Kinesis Data Analytics	Real-time SQL on streams	Flink or SQL
DataSync	Online data transfer to AWS	Agent on-prem, bandwidth throttling
Transfer Family	SFTP/FTPS/FTP to S3/EFS	Legacy protocol migration
Glue	Serverless ETL, Data Catalog	Crawlers discover schema
Glue Data Catalog	Central metadata	Used by Athena, EMR, Redshift
EMR	Managed Hadoop/Spark	Large-scale big data processing
Lake Formation	Data lake security	Central permissions, fine-grained
Athena	Serverless SQL on S3	Pay per query, no clusters



Section 3.5 Summary - Data Ingestion & Transformation

Data Pipeline Patterns (Exam Favorites):

Pattern	Services
Real-time dashboard	Kinesis → KDA → S3 → QuickSight
Data lake ingestion	Kinesis Firehose → S3 → Glue → Athena
ETL for warehouse	S3 → Glue → Redshift
Legacy FTP migration	Transfer Family → S3 → Glue → Redshift





GOOD JOB !!
TIME TO
TAKE A BREAK